百人搖搖競速小遊戲：系統架構與開發規劃

100 人高併發 × 防延遲公平競賽 × Go 單一執行檔極簡部署

- 玩法核心：玩家使用個人手機搖晃，將 G-Sensor 震動次數轉化為大螢幕角色前進速度
- 目標規模：單場支援接近 100 人同時連線競賽
- 手機零安裝：掃描大螢幕 QR Code 打開瀏覽器即可加入
- 跨網路絕對公平：消除不同電信商 4G/5G 網路延遲（Latency）與掉包誤差
- 極簡單一執行檔部署： Go (//go:embed) 封裝後端與大螢幕，丟上 Box PC 直接執行

1. 遊戲模式矩陣（開賽前 2 × 2 自由切換）

主持人可於開賽前隨時切換「參賽形式」與「獲勝條件」，玩家無須重掃 QR Code

- 【模式 1】個人賽 × 限時制（如 30 秒）：時間結束累積搖晃次數最高者勝，即時顯示 Top 15 英雄榜
- 【模式 2】個人賽 × 競速制（如先達 200 下）：先達終點者依手機端實際完賽毫秒數 (elapsedMs) 排名
- 【模式 3】分隊賽 × 限時制（如 30 秒）：自動 / 隨機分 4 隊（紅藍綠黃），以「隊伍平均每人搖晃數」推進
- 【模式 4】分隊賽 × 競速制（如平均達 200 下）：隊伍平均率先突破 200 下者獲勝
- 分隊公平性與榮譽感：以「平均搖晃數」消除人數差 1~2 人之不公，賽後同時頒發「全場個人 MVP 前 5 名」

2. 完整系統架構圖（含 Cloudflare Tunnel）

從手機端 G-Sensor 、 Cloudflare 安全穿透通道、到 Box PC 單一 Go 執行檔之完整拓樸

📱 100 支玩家手機（/play < 10KB）

- 各自透過 4G/5G 連線，免擠現場 Wi-Fi
- 網頁讀取硬體 G-Sensor (DeviceMotion 60Hz)
- iOS 點擊授權 + 80ms 防抖動 + 本地計時器

⬇ 每 150ms 節流傳送 WSS: { totalShakes, elapsedMs } ⬇

☁ Cloudflare Edge + Tunnel 加密穿透層

- 自動配發合法 HTTPS/SSL 憑證 (https://xxxx.trycloudflare.com)，手機不跳安全警告
- 由內往外主動建立加密通道 (Outbound)，完全免開公司防火牆 Port / 免買網域

⬇ 安全轉發至本機 127.0.0.1:3000 ⬇

💻 Linux Box PC — 單一 Go 執行檔 (racing)

① 通道管理：背景自動拉起 cloudflared 並擷取網址轉 QR Code
② go:embed ：內嵌 /play 與 /host 網頁資源（外部 0 零散檔案）
③ Go 高併發引擎：維護 2x2 狀態機、 15s 心跳、 10Hz 批次廣播

🖥️ 大螢幕全螢幕視窗（/host）

- Go 自動以 Chromium --app 喚起
- 自動解鎖音效播放、顯示 60FPS 賽道
- 關閉視窗即連動關閉 Server 與 Tunnel

3. 網路架設比較： Cloudflare Tunnel vs 公司自架外網 Server

為什麼 Cloudflare Tunnel 能同時解決 HTTPS 憑證、防火牆與外場移動問題？

- 【HTTPS 憑證問題】：純 IP 無法申請免費受信任憑證；自架需買網域 + 設 DNS + 配置 Let's Encrypt
- • Cloudflare Tunnel 直接配發合法 https:// 憑證，手機掃碼即玩，零安全警告視窗
- 【公司防火牆設定】：自架外網 Server 必須請網管設定路由器 Port Forwarding (轉發 80/443 Port)
- • Cloudflare Tunnel 由 Box PC 由內往外主動連線，完全不用動公司防火牆任何設定
- 【外場活動攜帶性】：若帶去飯店、餐廳辦尾牙，現場不可能讓你改飯店路由器； Tunnel 連手機熱點隨插即用
- 【100 人分流優勢】：玩家直接走各自 4G/5G 電信網路，徹底避開單一現場 Wi-Fi 分享器被 100 人塞爆

4. 手機網頁如何偵測 G-Sensor（加速度計）數值？

免安裝 App，透過 W3C 標準 DeviceMotionEvent API 直接讀取三軸物理加速度

- 原生支援： iOS Safari 、 Android Chrome 與 LINE 內建瀏覽器皆支援，每秒回傳約 60 次 (60Hz) 感測數值
- 主要數據 — event.acceleration (x, y, z)：已扣除地球重力 (9.8 m/s²) 之純移動加速度
- • 手機靜止時約為 (0, 0, 0)；手部一搖晃，合成向量瞬間飆升至 15 ~ 50 m/s²
- 備援數據 — event.accelerationIncludingGravity (x, y, z)：含地球重力之原始 G-Sensor 數值
- • 針對無陀螺儀之舊款平價手機，自動改算相鄰兩次差值 Δa ，確保 100% 機型皆能精準偵測
- 瀏覽器兩大強制安全條件：
- 1. 網址必須為 HTTPS （若為 http:// 瀏覽器會將 G-Sensor 數值全部鎖死為 null ）
- 2. iOS 13+ 必須在玩家「點擊準備按鈕 (click) 」瞬間呼叫 DeviceMotionEvent.requestPermission() 取得授權

5. 關鍵技術（1）：如何徹底消除網路延遲（Latency）不公？

確保網路延遲 30ms 與 300ms 的玩家擁有完全相同的獲勝公平性

- 【機制 1】傳送「累積總數 (totalShakes)」而非「單次 +1」：
- • 即使 4G 瞬間卡頓 1 秒掉封包，下一包送達即瞬間補齊所有累計次數，一下都不會少算
- 【機制 2】手機端本地精準計時 (elapsedMs)：
- • 限時模式下，手機本地嚴格鎖定精準 30.00 秒感測視窗；競速模式下，記錄本地完成之精確毫秒數
- 【機制 3】賽末 1.5 秒「衝線結算緩衝期 (Grace Period)」：
- • 時間到時大螢幕播放 1.5 秒衝線結算動畫，等待慢速網路最後一包數據收齊後再行公布名次

6. 關鍵技術（2）：硬體感測標準化與防塞車節流

消除 iPhone / Android 高低階硬體差異，並防止百人封包風暴

- 跨廠牌感測器標準化：高階手機 G-Sensor 取樣率達 100~120Hz ，舊機僅 50~60Hz
- 80ms 防抖動冷卻 (Debounce)：計算三軸加速度變化向量 Δa 超過門檻，且強制間隔 80ms 以上才計 1 下
- 符合人體極限：人類手部極限約每秒 8~10 下，鎖定上限 12.5 下 / 秒可確保所有手機計數基準一致
- 手機端上傳節流 (150ms)：本地即時累加，每 150ms 傳送一次， 100 人每秒合計僅約 660 個極小封包
- 伺服器廣播節流 (100ms / 10Hz)： Go 伺服器每 100ms 統一打包廣播給大螢幕，大螢幕以 60 FPS 平滑插值

7. 關鍵技術（3）：百人活動現場的 4 大實戰防護

針對百人現場可能發生的突發狀況預先設計防呆機制

- 防 Cloudflare 瞬間限流（HTTP 429）：手機網頁採單一內聯 HTML 並開 Gzip (< 10KB)，零外部圖片請求
- 防待機斷線與重連恢復： 15 秒雙向 Heartbeat + WakeLock 防螢幕變暗 + localStorage ID 斷線秒回原角色
- 等候室「熱身試搖」自檢：開賽前玩家搖手機，大螢幕上其名字即跳動發光，開賽前即可確認 G-Sensor 正常
- iOS 搖晃復原防呆：玩家輸入完暱稱按下準備瞬間，程式自動解除輸入框焦點（blur），避開 iOS 系統彈窗

8. 硬體選型與單一執行檔自動化部署

免安裝 Node.js 、免開終端機複製網址、免手動開瀏覽器

- 硬體推薦首選： i5-6100U Box PC (Ubuntu / Debian Desktop)
- • Go 後端在 100 人併發下 CPU 佔用率 < 3% ； Intel x86 內顯對 Chromium 硬體加速最成熟穩定 (60 FPS)
- • 亦支援透過 Go 一鍵跨平台編譯至 ARM64 (RK3588) 與 Windows (.exe)
- 執行單一檔案 ./racing 自動完成三步驟：
- 1. 啟動內嵌 WebSocket 與網頁伺服器
- 2. 背景自動啟動 cloudflared 並擷取 HTTPS 網址轉為大螢幕 QR Code
- 3. 自動帶入 Kiosk 參數 (--app 全螢幕 + 解鎖音效自動播放) 彈出視窗，關閉視窗即自動清理背景行程

9. 開發階段與里程碑規劃（Roadmap）

分階段實作與百人虛擬機器人壓力測試

- 【Phase 1】 Go 核心伺服器與自動化通道： WebSocket 房間管理、 15s 心跳、 10Hz 廣播、 cloudflared 自動整合
- 【Phase 2】手機控制器與 G-Sensor 防延遲演算法： < 10KB 內聯網頁、 iOS 授權、 80ms 防抖動、本地計時、斷線重連
- 【Phase 3】大螢幕主機端與 2×2 四模式規則：開賽前模式切換、熱身試搖同步、馬拉松 / 四隊賽道、 1.5s 結算緩衝
- 【Phase 4】百人機器人壓測與跨平台封裝：內建「一鍵模擬 100 名 Bot 玩家」壓測工具、打包 Windows/Linux 執行檔